

Upgrade Correction 1: Estate Modules from Odoo 16 to Odoo 19

This correction follows the files, in their order of appearance.

`estate/__manifest__.py`

Pitfalls

- The module version is still `16.0.1.0.0`.
- The module loads the legacy JavaScript asset
`estate/static/src/js/legacy_property_widget.js`.
- Dependencies must be checked against the Odoo 19 addon set.

Correction

Set the first target-version module release to:

```
'version': '19.0.1.0.0',
```

Keep `mail` and `hr_holidays` only if the converted module still needs their models and behavior. After converting or removing the JavaScript widget, update the asset list. Confirm that every data, security, and test file still loads.

`estate/controllers/main.py`

Pitfall: deprecated JSON route type

The routes use the v16-style declaration:

```
@http.route('/estate/properties', type='json', ...)
```

In Odoo 19, `type='json'` is a deprecated alias.

Correction

Use the explicit JSON-RPC route type:

```
@http.route(  
    '/estate/properties',  
    type='jsonrpc',  
    auth='user',  
    methods=['POST'],
```

```
        csrf=False,  
    )
```

Retest the request and response format with a real JSON-RPC client. Recheck authentication, access rights, input validation, error responses, and whether `csrf=False` is genuinely necessary.

`estate/models/estate_property.py`

Pitfall: Python field `states`

`commission_rate` uses the old field-state definition:

```
states={'sold': [('readonly', True)], 'cancelled': [('readonly', True)]}
```

Correction

Remove `states` from the field and express the UI behavior in XML:

```
commission_rate = fields.Float(  
    string="Commission Rate (%)",  
    default=6.0,  
)
```

```
<field name="commission_rate" readonly="state in ['sold', 'cancelled']"/>
```

Pitfall: `mail.channel`

The model contains:

```
channel_id = fields.Many2one('mail.channel')
```

The modern Discuss model is `discuss.channel`.

Correction

Change the relation and all related Python, XML, security, integration, automated-action, and external-ID references:

```
channel_id = fields.Many2one(  
    'discuss.channel',
```

```
string="Discussion Channel",
)
```

Write a migration for existing relation values. Determine the mapping between old and new channel records before changing the field! Do not simply discard the relation.

Pitfall: `name_get`

The model overrides the legacy naming method:

```
def name_get(self):
    ...
```

Correction

Use `_compute_display_name`:

```
@api.depends('name', 'postcode')
def _compute_display_name(self):
    for record in self:
        postcode = record.postcode or 'no postcode'
        record.display_name = f"{record.name} ({postcode})"
```

Verify Many2one dropdowns and search results.

Pitfall: raw SQL against `ir_property`

`get_legacy_company_commission()` queries the `ir_property` table directly. Company-dependent values moved to JSONB storage on model tables in newer versions.

Correction

Use ORM access:

```
return self.commission_rate
```

If the field should be company-dependent, define that through the supported field option and continue reading and writing it through the ORM. Do not rely on a storage table layout. Migrate historical values with a versioned upgrade script.

Pitfall: `fields_view_get`

The model contains the v16 view API:

```
def fields_view_get(self, view_id=None, view_type='form', toolbar=False,
                    submenu=False):
    ...
```

Odoo 19 uses `get_view/get_views`; opening a view can fail if the old method remains.

Correction

Remove the override if it performs no real work. If dynamic processing is required, migrate it:

```
@api.model
def get_view(self, view_id=None, view_type='form', **options):
    return super().get_view(
        view_id=view_id,
        view_type=view_type,
        **options,
    )
```

Tip: Only modify the returned architecture when there is a functional requirement.

`estate/models/estate_property_offer.py`

Improvement: single-record `create`

The override uses:

```
@api.model
def create(self, vals):
    ...
```

This is a single-record implementation and is unsafe or inefficient when the ORM creates several offers in one call. Not needed, but strongly recommended,

Correction

Use the batch API and validate before applying state changes:

```
@api.model_create_multi
def create(self, vals_list):
    properties = self.env['estate.property'].browse(
        [vals.get('property_id') for vals in vals_list]
    )
    if properties.filtered(lambda property: property.state == 'sold'):
        raise UserError(_("Cannot create an offer for a sold property"))

    records = super().create(vals_list)
```

```
properties.write({'state': 'offer_received'})
return records
```

Keep the controller contract as one offer if desired; a one-item list is still handled correctly by `model_create_multi`. Discuss looping over records, vs ensemble operations.

Improvement: incorrect `@api.depends`

`action_offer_accept()` is a button method but is decorated with `@api.depends`. It is not a v19 removal, but it incorrectly presents the method as a computed-field method.

Correction

Remove the decorator:

```
def action_offer_accept(self):
    ...
```

Use `@api.depends` only for methods computing declared computed fields.

`estate/models/hr_leave.py`

Pitfall: `_force_cancel` signature

The override accepts only:

```
def _force_cancel(self, reason, msg_subtype="mail.mt_comment"):
```

Newer callers may pass additional arguments, including `notify_responsibles`.

Correction

Mirror the target-version signature and forward the arguments:

```
def _force_cancel(
    self,
    reason=None,
    msg_subtype="mail.mt_comment",
    notify_responsibles=True,
):
    result = super()._force_cancel(
        reason,
        msg_subtype=msg_subtype,
        notify_responsibles=notify_responsibles,
    )
```

```
# Preserve the custom notification behavior.  
return result
```

Test every leave-cancellation workflow and ensure notifications are not duplicated.

estate/views/estate_property_views.xml

Pitfall: `attrs` and `states`

Legacy modifiers such as:

```
attrs="{ 'invisible': [('garden', '=', False)] }"
```

are no longer accepted in the newer view architecture.

Correction

Use inline expressions:

```
invisible="not garden"  
readonly="state in ['sold', 'cancelled']"
```

estate/views/estate_property_offer_views.xml

Pitfall

Review this view for legacy `attrs/states` modifiers and convert them to inline `invisible`, `readonly`, or `required` expressions.

Correction

For example:

```
<field name="is_expired" invisible="status != 'accepted'"/>
```

estate/views/estate_property_type_views.xml

Pitfall: `tree` renamed to `list`

Convert view roots, nested One2many views, and action modes:

```
<tree>...</tree>
```

becomes:

```
<list>...</list>
```

and:

```
view_mode="tree, form"
```

becomes:

```
view_mode="list, form"
```

Also update `kanban, tree, form` to `kanban, list, form`.

Pitfall: nested `column_invisible`

The nested view uses a condition based on a parent field without `parent.:`

```
<field name="expected_price" column_invisible="offer_count == 0"/>
```

Correction

Reference the parent explicitly:

```
<field
  name="expected_price"
  column_invisible="parent.offer_count == 0"/>
```

`column_invisible` controls the whole column, not an individual cell.

Pitfall: stale XPath target

An inherited view targets:

```
//field[@name='property_ids']/tree/field[@name='expected_price']
```

This no longer matches after converting the nested view.

Correction

Use a specific target matching the new architecture:

```
<xpath
  expr="//field[@name='property_ids']/list/field[@name='expected_price']"
  position="after">
  <field name="state" string="Estate Status"/>
</xpath>
```

Validate every XPath against the actual Odoo 19 parent architecture and ensure that it matches exactly one intended node.

`estate/static/src/js/legacy_property_widget.js`

Pitfall

The file uses the legacy AMD and widget APIs:

```
odoo.define('estate.legacy_property_widget', function (require) {
  const Widget = require('web.Widget');
  return Widget.extend({...});
});
```

The legacy widget system was replaced by Owl-based frontend code.

Correction

If the widget is no longer needed, remove the asset and its markup. Otherwise:

- Convert it to an Owl component.
- Move initialization into `setup()`.
- Replace `start()` with appropriate hooks such as `onMounted`.
- Register the component or field widget with the v19 registry.
- Convert templates and update the asset bundle.
- Test rendering, interaction, and teardown in the browser.

Do not merely rename `start()` to `onMounted`; the component lifecycle and registration model are different.

`estate/tests/common.py`

Status

The fixture was changed to create offers individually for the intentional single-record trap. After correcting `create()` to `model_create_multi`, update the fixture to exercise both single-record and batch creation.

`estate/tests/test_estate_legacy.py`

Pitfall: `SavepointCase`

The test imports and inherits from the v16-era base:

```
from odoo.tests.common import SavepointCase
```

This is not the supported v19 test base.

Correction

Convert it to `TransactionCase`:

```
from odoo.tests import TransactionCase

class TestEstateLegacyApi(TransactionCase):
    ...
```

`estate_account/__manifest__.py`

Improvement

The module version remains on the Odoo 16 version:

```
'version': '16.0.1.0.0',
```

Correction

Set it to the first Odoo 19 release:

```
'version': '19.0.1.0.0',
```

`estate_account/tests/test_estate_property.py`

`expression` can now be turned into Domain, by importing the `odoo.fields.Domain` class